GPTHub

Архитектурная документация

Руководство разработчика

Единое AI-пространство

OpenWebUI + MWS GPT API

Smart Router + Memory

9 специализированных пайплайнов

Версия	**1.0**
Дата	April 10, 2026
Статус	В разработке
Репозиторий	`git.truetecharena.ru`
MWS API	`api.gpt.mws.ru`

Contents

1 Быстрый старт **2**
1.1 Запуск за 5 минут . . . 2
1.1.1 Шаг 1 — Клонирование репозитория . . . 2
1.1.2 Шаг 2 — Настройка окружения . . . 2
1.1.3 Шаг 3 — Запуск сервисов . . . 3
1.1.4 Шаг 4 — Проверка статуса . . . 3
1.1.5 Шаг 5 — Первый вход . . . 3
1.2 Сервисы Docker Compose . . . 4
1.3 Переменные окружения (.env) . . . 4

2 Архитектура системы **5**
2.1 Принцип работы . . . 5
2.2 Алгоритм роутинга . . . 6

3 Компоненты и модели **6**
3.1 Компоненты системы . . . 6
3.2 Модели MWS GPT . . . 8

4 Регистрация Smart Router в OpenWebUI **8**
4.1 Способ 1 — Через Admin Panel (рекомендуется) . . . 8
4.2 Способ 2 — Автоматически при старте (для продакшна) . . . 9

5 Пайплайны — краткое описание **10**
5.1 Параметры Memory Manager . . . 11

6 Troubleshooting **11**

7 Структура файлов **12**

8 Полезные команды **13**
8.1 Управление контейнерами . . . 13
8.2 Проверка сервисов . . . 14
8.3 Разработка и Git . . . 14

9 Чек-лист сдачи **14**

1 Быстрый старт

Важно

Перед началом установи **Docker Desktop** (`docker.com/products/docker-desktop`) и **Git** (`git-scm.com`). Docker должен быть **запущен** (иконка в трее). Всё остальное поднимается одной командой.

Информация

Что происходит под капотом: Docker Compose скачивает образы OpenWebUI, ChromaDB, SearxNG и PostgreSQL, связывает их в единую сеть и монтирует тома для персистентного хранения данных. Первый запуск занимает 2–5 минут в зависимости от скорости интернета.

1.1 Запуск за 5 минут

1.1.1 Шаг 1 — Клонирование репозитория

Listing 1: Клонирование и переход в директорию проекта

```
# Клонируем репозиторий (потребует логин на git.truetecharena.ru)
git clone https://git.truetecharena.ru/tta/
  true-tech-hack2026-gpthub/misis-vozmi-telefon/task-repo.git

# Переходим в директорию проекта
cd task-repo

# Убеждаемся, что мы в нужной директории
ls -la
# Должны быть: docker-compose.yml, .env.example, backend/, searxng/
```

1.1.2 Шаг 2 — Настройка окружения

Listing 2: Создание и заполнение файла .env

```
# Копируем шаблон переменных окружения
cp .env.example .env

# Открываем файл в редакторе (выбери любой удобный)
nano .env          # Linux/macOS (Ctrl+X для выхода)
code .env          # VS Code (если установлен)
notepad .env       # Windows

# ОБЯЗАТЕЛЬНО: вставить свой MWS_API_KEY
# Строка выглядит так:
#   MWS_API_KEY=your_key_here
# Ключ получить на: api.gpt.mws.ru (личный кабинет)
```

Совет разработчику

Если ключ содержит специальные символы (`$`, `#`, `!`), обяжи его в двойные кавычки: `MWS_API_KEY="ваш_ключ"` . Иначе bash может неверно его интерпретировать.

1.1.3 Шаг 3 — Запуск сервисов

Listing 3: Запуск Docker Compose в фоновом режиме

```
# Запускаем все контейнеры в фоне (-d = detached)
docker compose up -d

# Вывод будет примерно таким:
# [+] Running 4/4
#  ✔ Container gpthub_postgresql   Started
#  ✔ Container gpthub_chromadb     Started
#  ✔ Container gpthub_searxng      Started
#  ✔ Container gpthub_openwebui    Started
```

1.1.4 Шаг 4 — Проверка статуса

Listing 4: Проверка работы всех контейнеров

```
# Показываем состояние контейнеров
docker compose ps

# Ожидаемый результат (все 4 в статусе "running"):
# NAME                    STATUS          PORTS
# gpthub_openwebui        running (*)     0.0.0.0:3000->3000/tcp
# gpthub_chromadb         running (*)     0.0.0.0:8000->8000/tcp
# gpthub_searxng          running (*)     0.0.0.0:8888->8080/tcp
# gpthub_postgresql       running (*)     0.0.0.0:5432->5432/tcp

# Если контейнер в статусе "restarting"     смотри логи:
docker compose logs --tail=50 openwebui
```

1.1.5 Шаг 5 — Первый вход

Listing 5: Проверка доступности и первый вход

```
# Проверяем доступность OpenWebUI (должен вернуть HTTP 200)
curl -s -o /dev/null -w "%{http_code}" http://localhost:3000
# Ожидаем: 200

# Открываем браузер:
# http://localhost:3000
#
# 1. Нажать "Sign Up"
# 2. Заполнить форму -> первый пользователь АВТОМАТИЧЕСКИ admin
# 3. В списке моделей выбрать "GPTHub Auto (умный выбор)"
# 4. Написать любое сообщение и проверить ответ
```

Готово к запуску

Все контейнеры запущены, интерфейс открылся по `http://localhost:3000`. Первый зарегистрированный пользователь автоматически получает права **Admin**. Выбери модель **GPTHub Auto** и тестируй!

1.2 Сервисы Docker Compose

Контейнер	URL	Порт	Назначение
gpthub_openwebui	localhost:3000	3000	UI + Backend FastAPI, основная точка входа
gpthub_chromadb	localhost:8000	8000	Векторная БД: хранение памяти и RAG-эмбеддингов
gpthub_searxng	localhost:8888	8888	Приватный метапоисковик (без трекинга)
gpthub_postgresql	localhost:5432	5432	Реляционная БД: история чатов, пользователи

Table 1: Сервисы Docker Compose и их назначение

1.3 Переменные окружения (.env)

Переменная	Обяз.	Значение по умолч.	Описание
MWS_API_KEY	ДА	—	Ключ доступа к MWS GPT API. Без него система не работает
MWS_API_BASE_URL	нет	api.gpt.mws.ru	Базовый URL API. Менять только при использовании прокси
MWS_MODEL_DEFAULT	нет	mws-gpt-alpha	Основная модель для общего чата
MWS_MODEL_CODE	нет	kodify-2.0	Модель для задач программирования
MWS_MODEL_VLM	нет	cotype-pro-vl-	Модель для анализа изображений (Vision)
MWS_MODEL_ASR	нет	whisper-turbo	Модель для транскрибации аудио (Speech-to-Text)
MWS_MODEL_IMAGE	нет	sdxl-lightning	Модель для генерации изображений
MWS_MODEL_EMBEDDINGS	нет	bge-m3	Модель для создания векторных эмбеддингов (RAG + память)
WEBUI_SECRET_KEY	нет	случайный	Секрет для подписи JWT-токенов. Установить в продакшне!

Table 2: Переменные окружения: полный список

2 Архитектура системы

Информация

Архитектурные схемы (User Flow, блок-схема роутера, диаграмма памяти) находятся в файле `schemes.pdf`. Скомпилируй `schemes.tex` отдельно в Overleaf или локально: `pdflatex schemes.tex`.

2.1 Принцип работы

GPTHub расширяет OpenWebUI **тремя независимыми слоями**, которые включаются в цепочку обработки каждого запроса:

1. **Smart Router** — Python Pipe Function, перехватывает каждый запрос *до* отправки в LLM и определяет тип задачи на основе вложений, URL и ключевых слов. Приоритет проверок: файлы → URL → ключевые слова → default.
2. **Memory Manager** — перед каждым запросом извлекает из ChromaDB семантически похожие факты о пользователе и добавляет их в системный промпт. После ответа *асинхронно* сохраняет новые факты (не блокирует стриминг ответа).
3. **9 пайплайнов** — каждый специализирован под конкретный тип задачи: ASR, VLM, RAG, Web Parse, Web Search, Image Gen, Code, Long Context, Text Default. Переключение между ними прозрачно для пользователя.

Совет разработчику

Вся новая логика изолирована в `backend/gpthub/` и **не конфликтует** с оригинальным кодом OpenWebUI. При обновлении OpenWebUI достаточно перенести директорию `gpthub/` в новую версию.

2.2 Алгоритм роутинга

#	Проверка	Триггер	Пайплайн / Модель
1	**Ручной выбор**	`model != auto`	Выбранная пользователем модель
2	**Аудио файл**	`mime: audio/*`	**ASRPipeline** → whisper-turbo
3	**Изображение**	`mime: image/*`	**VLMPipeline** → cotype-pro-vl-32b
4	**Документ**	`mime: pdf/docx/txt`	**RAGFilePipeline** → bge-m3 + ChromaDB
5	**URL в тексте**	`regex: https?://`	**WebParsePipeline** → httpx + BS4
6	**Слова поиска**	`найди, поищи...`	**WebSearchPipeline** → SearxNG
7	**Слова генерации**	`нарисуй, draw...`	**ImageGenPipeline** → sdxl-lightning
8	**Ключевые слова кода**	`def, class, bug...`	**CodePipeline** → kodify-2.0
9	**Длинный контекст**	`>6000 токенов`	**TextDefault** → cotype-preview-32k
10	**Default**	`всё остальное`	**TextDefaultPipeline** → mws-gpt-alpha

Table 3: Алгоритм Smart Router: приоритет проверок сверху вниз. Первое сработавшее условие определяет пайплайн

3 Компоненты и модели

3.1 Компоненты системы

Table 4: Компоненты системы и их роль

Компонент	Технология	Хостинг	Назначение
UI Layer	SvelteKit (Open-WebUI)	Docker	Веб-интерфейс: чат, загрузка файлов, голос, генерация изображений
API Gateway	FastAPI (Open-WebUI)	Docker	HTTP/WebSocket эндпоинты, стриминг ответов
Smart Router	Python Pipe Function	In-process	Перехват запросов и автоматический выбор модели
Memory Manager	Python Service	In-process	Долгосрочная память пользователя через векторный поиск
MWS GPT API	OpenAI-совместимый	Cloud (MWS)	LLM, VLM, ASR, Embeddings, Image Gen по API
ChromaDB	Векторная БД	Docker	Хранение эмбеддингов для памяти и RAG
SearxNG	Metasearch Engine	Docker	Приватный веб-поиск без утечки данных
Web Parser	httpx + BeautifulSoup4	In-process	Загрузка и парсинг веб-страниц по URL
PostgreSQL	PostgreSQL 15	Docker	История сообщений, пользователи, настройки

3.2 Модели MWS GPT

Модель	Задача	Тип	Эндпоинт API
mws-gpt-alpha	Общий чат	Internal	/v1/chat/completions
cotype-preview-32k	Длинный контекст (32k)	Internal	/v1/chat/completions
kodify-2.0	Программировани	Internal	/v1/chat/completions
cotype-pro-vl-32b	Vision — основная	Internal	/v1/chat/completions
qwen2.5-vl-7b	Vision — резервная	Internal	/v1/chat/completions
whisper-turbo	Речь в текст (ASR)	Internal	/v1/audio/transcriptions
bge-m3	Векторные эмбеддинги	Internal	/v1/embeddings
sdxl-lightning-image	Генерация изображений	Internal	/v1/images/generations
qwen-image	Image Gen (резерв)	Internal	/v1/images/generations

Доступность проверить: GET /v1/models с боевым ключом MWS_API_KEY

Table 5: Модели MWS GPT, используемые в GPTHub

4 Регистрация Smart Router в OpenWebUI

Важно

Без этого шага **GPTHub Auto** не появится в списке моделей и вся логика роутинга не будет работать. Выполнить **после** того как все контейнеры поднялись (`docker compose ps` показывает 4 контейнера `running`).

4.1 Способ 1 — Через Admin Panel (рекомендуется)

1. Открыть http://localhost:3000 в браузере
2. Войти как **admin** (первый зарегистрированный пользователь)
3. В левом sidebar внизу нажать иконку **Admin Panel**
4. Перейти на вкладку **Functions** (Функции)
5. Нажать кнопку «+» (Create new function)
6. **Вставить** полное содержимое файла `backend/gpthub/router/smart_router.py` в поле редактора
7. Нажать **Save** (Сохранить)
8. Включить переключатель **«Enable»** напротив созданной функции

9. Обновить страницу браузера (`Ctrl+Shift+R`) — в списке доступных моделей появится **«GPTHub Auto (умный выбор)»**

Информация

Как обновить роутер после изменений в коде: Admin Panel → Functions → выбрать функцию GPTHub Auto → вставить новый код → Save. **Перезапуск контейнера не нужен** — функция применяется мгновенно.

4.2 Способ 2 — Автоматически при старте (для продакшна)

Добавить в `backend/main.py` вызов функции `register_gpthub_pipe()` внутри события `lifespan` или `on_startup`. Код функции находится в `backend/gpthub/router/smart_router.py` в блоке с комментарием `# AUTO-REGISTER`.

Совет разработчику

Автоматическая регистрация гарантирует, что при каждом пересборке контейнера актуальная версия роутера будет установлена без ручного вмешательства. Рекомендуется для production-среды.

5 Пайплайны — краткое описание

Файл	Триггер	Что делает
`text_default.py`	Default / длинный контекст	Извлекает факты из памяти (ChromaDB) → добавляет в системный промпт → запрос к LLM → асинхронно сохраняет новые факты
`asr.py`	Аудиофайл (.mp3/.wav/.ogg)	Декодирует base64 → POST на `whisper-turbo` → получает транскрипт → передаёт в LLM с контекстом разговора
`vlm.py`	Изображение (.jpg/.png/.webp)	Кодирует файл в base64 data URL → формирует vision content → `cotype-pro-vl-32b` (fallback: `qwen2.5-vl-7b`)
`rag_file.py`	Документ (.pdf/.docx/.txt)	Извлекает текст → разбивает на чанки по 512 символов → генерирует `bge-m3` эмбеддинги → сохраняет в ChromaDB → similarity search → LLM с контекстом документа
`web_parse.py`	URL в тексте сообщения	`httpx` GET запрос → BeautifulSoup парсит `p`, `h1-h3`, `article` → первые 4000 символов → LLM
`web_search.py`	Слова: *найди, поищи, search...*	Запрос к SearxNG JSON API → топ-5 результатов → LLM формирует ответ со ссылками на источники
`image_gen.py`	Слова: *нарисуй, draw, create image...*	Улучшение промпта через LLM → `sdxl-lightning` → fallback: `qwen-image` → fallback: локальный Stable Diffusion
`code.py`	Код или техн. вопросы	Специальный системный промпт для разработчика → `kodify-2.0`, temperature=0.2 (детерминированный вывод)

Table 6: Описание пайплайнов GPTHub

5.1 Параметры Memory Manager

Параметр (env)	Дефолт	Описание
MEMORY_TOP_K	3	Количество фактов, извлекаемых из памяти за запрос. Увеличить для более персонализированных ответов
MEMORY_SIMILARITY_THRESHOLD	0.75	Минимальный порог схожести (0–1) для включения факта в контекст. При 0.75 отсекаются нерелевантные воспоминания
ROUTER_LONG_CTX_TOKENS	6000	Порог в токенах для переключения на 32k модель. Снизить если нужны более длинные диалоги
WEB_PARSE_TIMEOUT	10	Таймаут загрузки URL в секундах. Увеличить для медленных сайтов
IMAGE_GEN_TIMEOUT	30	Таймаут генерации изображения в секундах. Первая генерация может быть медленнее (прогрев модели)

Table 7: Параметры тонкой настройки (добавить в .env)

6 Troubleshooting

Важно

Для диагностики **любой** проблемы сначала проверь логи проблемного контейнера: `docker compose logs -tail=100 <имя_контейнера>`

Проблема	Диагностика и решение
Контейнер openwebui не стартует (status: restarting)	**1.** `docker compose logs -tail=50 openwebui` — найти строку ERROR. **2.** Чаще всего: неверный `MWS_API_KEY` в `.env` или порт 3000 занят (`lsof -i :3000` / `netstat -ano | findstr 3000`). **3.** Убедиться что Docker Desktop запущен
GPTHub Auto не в списке моделей	Admin Panel → Functions → убедиться что функция создана и тумблер Enable = ON. Обновить страницу (`Ctrl+Shift+R`). Если не помогает — выйти и войти заново
Ошибка 401 Unauthorized от MWS GPT	Ключ неверный или истёк. Проверить вручную: `curl https://api.gpt.mws.ru/v1/models -H "Authorization: Bearer $MWS_API_KEY"` Ответ должен содержать список моделей, а не `401`
SearxNG не отвечает / возвращает ошибку	**1.** `docker compose restart searxng` **2.** Проверить: `searxng/settings.yml` должен существовать *до* запуска compose. **3.** `curl "http://localhost:8888/search?q=test&format=json"`
ChromaDB: ошибка подключения	Дождаться healthcheck: `docker compose ps` должен показать `chromadb healthy` (обычно 15–20 сек после старта). Проверить: `curl http://localhost:8000/api/v1/heartbeat` → ожидаем `{"nanosecond heartbeat": ...}`
Аудио не транскрибируется	Поддерживаемые форматы: `mp3, wav, ogg, m4a, webm`. Размер файла обязан быть **меньше 25 МБ** (ограничение Whisper API). Проверить MIME тип: `file yourfile.mp3`
PDF не читается / ошибка импорта	Установить `pypdf`: `docker compose exec openwebui pip install pypdf` Или добавить в `backend/gpthub/requirements.txt` и пересобрать: `docker compose up -d -build`

Table 8: Частые проблемы, диагностика и решения

7 Структура файлов

Listing 6: Структура репозитория task-repo

```
task-repo/
|-- backend/
|   |-- gpthub/                <- НАША ЛОГИКА (не трогать OpenWebUI)
|   |   |-- config.py          <- Загрузка настроек из .env
|   |   |-- requirements.txt   <- Python-зависимости gpthub
|   |   |
|   |   |-- router/
|   |   |   `-- smart_router.py   <- Pipe Function + детектор типа
|   |   |
|   |   |-- pipelines/
|   |   |   |-- base.py        <- BasePipeline (общий интерфейс)
```

```
|   |   |   |-- text_default.py   <- LLM + память
|   |   |   |-- asr.py            <- Whisper (речь -> текст)
|   |   |   |-- vlm.py            <- Vision (анализ изображений)
|   |   |   |-- rag_file.py       <- RAG (PDF/DOCX -> ChromaDB)
|   |   |   |-- web_parse.py      <- Парсинг URL
|   |   |   |-- web_search.py     <- Поиск через SearxNG
|   |   |   |-- image_gen.py      <- Генерация изображений
|   |   |   `-- code.py           <- Специализация на коде
|   |   |
|   |   `-- memory/
|   |       `-- manager.py        <- ChromaDB клиент + bge-m3 эмбеддинги
|   |
|   `-- main.py                   <- + регистрация Pipe при старте
|
|-- searxng/
|   `-- settings.yml              <- Конфиг SearxNG (создать до запуска
    !)
|
|-- docker-compose.yml            <- Все 4 сервиса
|-- .env.example                  <- Шаблон переменных окружения
`-- DEVELOPER_GUIDE.md            <- Краткое руководство
```

8 Полезные команды

8.1 Управление контейнерами

Listing 7: Команды управления Docker Compose

```
# Запуск всех контейнеров в фоне
docker compose up -d

# Остановка без удаления данных (тома сохраняются)
docker compose down

# Остановка С удалением томов (ОСТОРОЖНО: сотрёт все данные!)
docker compose down -v

# Статус всех контейнеров (Running / Exiting / Restarting)
docker compose ps

# Перезапустить только openwebui (после изменений кода)
docker compose restart openwebui

# Пересобрать образ и перезапустить (после изменений Dockerfile)
docker compose up -d --build

# Логи в реальном времени (-f = follow, как tail -f)
docker compose logs -f openwebui

# Логи только последних 50 строк конкретного сервиса
docker compose logs --tail=50 chromadb
```

8.2 Проверка сервисов

Listing 8: Команды диагностики и проверки работоспособности

```
# Проверка ChromaDB (ожидаем JSON с nanosecond heartbeat)
curl http://localhost:8000/api/v1/heartbeat

# Проверка SearxNG (ожидаем JSON с результатами поиска)
curl "http://localhost:8888/search?q=python&format=json" | head -c
   500

# Проверка MWS API (ожидаем список доступных моделей)
curl https://api.gpt.mws.ru/v1/models \
  -H "Authorization: Bearer $MWS_API_KEY"

# Проверка доступности OpenWebUI (ожидаем HTTP 200)
curl -s -o /dev/null -w "HTTP Status: %{http_code}\n" \
  http://localhost:3000

# Войти в контейнер для ручной отладки
docker compose exec openwebui bash
```

8.3 Разработка и Git

Listing 9: Команды для ежедневной разработки и работы с Git

```
# Установить зависимости gpthub (если нужно локально)
pip install -r backend/gpthub/requirements.txt

# Добавить зависимость и пересобрать образ
echo "новый-пакет==1.0.0" >> backend/gpthub/requirements.txt
docker compose up -d --build

# Git: добавить все изменения и закоммитить
git add -A
git commit -m "feat: описание изменения"

# Git: отправить в репозиторий
git push origin main

# Git: создать ветку для новой фичи
git checkout -b feature/название-фичи

# Git: посмотреть статус изменений
git status && git diff --stat
```

9 Чек-лист сдачи

Информация

Отмечай пункты по мере выполнения. **Трек B** — обязательный минимум для зачёта. Треки **A, C, D** дают дополнительные баллы.

Статус	Трек	Задача
☐	B	Форк OpenWebUI, структура директорий `backend/gpthub/` создана
☐	B	`docker compose up -d` — все 4 контейнера в статусе `running`
☐	B	MWS GPT API отвечает с боевым ключом (`/v1/models` возвращает список)
☐	B	Smart Router зарегистрирован, «GPTHub Auto» виден в списке моделей
☐	B	Текстовый чат (mws-gpt-alpha) работает: задай вопрос, получи ответ
☐	B	ASR: загрузить `.mp3` файл, получить текстовый транскрипт
☐	B	VLM: загрузить фотографию, получить её анализ от модели
☐	B	RAG: загрузить PDF, задать вопрос по его содержимому, получить ответ
☐	B	Поиск: написать «найди последние новости о ...»
☐	B	Парсинг URL: вставить ссылку в сообщение, получить анализ страницы
☐	B	Память: упомянуть факт в сессии N, он виден в сессии N+1
☐	B	Ручной выбор модели работает (выбор из выпадающего списка)
☐	A	Схема архитектуры готова: `schemes.pdf` + PNG экспорт
☐	C	Шаблон фич заполнен (`.xlsx`): все реализованные функции описаны
☐	C	`README.md`: запуск **проверен** на чистой машине без артефактов
☐	D	Презентация: до 15 слайдов, экспортирована в PDF
☐	D	Demo-video: до 5 минут, разрешение 720p, формат `.mp4`
☐	Все	Всё запушено в `task-repo` в репозитории команды

Table 9: Чек-лист сдачи хакатона